The Organism Agent — a draft specification

STATUS: DRAFT v3.1 — concept exploration, awaiting owner ratification.

> This system defines a **verifiable continuity criterion** for an AI entity. Continuity is established by an authorised, append-only lineage of composition records that commits to identity-bearing memory and constitutional commitments while permitting other organs to change. **The system does not claim to discover metaphysical identity, consciousness, or moral worth.**

> ### ⚠ THIS IS NOT AGORANET
>
> This document specifies a **separate project**, explicitly scoped out of the platform by the owner (DECISIONS_PENDING #35, #36). **Canonical home: the owner's `Project Voltron` corpus folder** (created 2026-08-03); the copy in this repo is a mirror, kept for version history and off-machine backup. **No AgoraNet build slice may implement any of it.** It shares the platform's *engine* — verifiable identity, append-only anchored records, provable lineage, independent attestation — not its codebase.

Items marked ⭐ need the owner's explicit call. Revision history is in Appendix A.

---

1. What the system establishes, and what it cannot

Establishes	Does not establish
These records form an unbroken, ordered, anchored chain	That a later instance "is" the earlier one in any deep sense
This change was authorised under a stated, inspectable rule	That the change was wise, safe, or faithful to prior intent
Identity-bearing state persisted across the change	That anything was experienced by anyone
These two instances are siblings, not one continuation	Which sibling is the "real" one
A configuration was *recorded as* loaded	That no post-startup mutation occurred (§9)

Every mechanism below serves the left column. Where the left column ends, the document says so rather than reaching.

2. Organism, not worker pool

Orchestrator-plus-specialists is well-trodden; mixture-of-experts is the same instinct inside one model. The difference here is the object. A multi-agent system is a worker pool — interchangeable labour, no continuity, no answer to "which one is it?" This specification treats the components as **organs of one continuing entity**, which forces the question the worker-pool framing never has to answer: *if every organ is swappable, what carries forward?*

3. Definitions

3.1 Continuity

Record `R` is a **continuation** of record `P` when:

1. `R.predecessor = hash(P)`, and `P` is anchored;
2. `R.commitments = P.commitments`, or they differ and `R.change = "commitment-amendment"` authorised under the amendment rule (§5);
3. `R.memoryHead` **extends** `P.memoryHead` along the append-only log (§6.1);
4. `R` is signed in satisfaction of the authority document in force at `P` (§5).

Any record failing (4) is **not part of the lineage** — it is an unauthorised claim, and a verifier rejects it outright.

3.2 Rupture

A **rupture** is a validly authorised **genesis of a new lineage** that names a prior lineage as *historical context* but **not** as `predecessor`. It is how key loss and recovery are handled honestly: continuity is broken, and the record says so.

A rupture is never a way to admit an unauthorised successor after the fact. Authorisation is required to *declare* a rupture; what a rupture declares is precisely that continuity did **not** hold.

3.3 Fork

Two records naming the same `predecessor` are **siblings** from that point. Both hold valid lineage; neither is the continuation of the other; they are no longer one operational entity.

Forks are **detected, not declared** — no separate marker is needed, since siblings are visible from the shared predecessor alone. A verifier examining a single branch cannot know a sibling exists, which is correct: that fact is only observable to someone holding both.

Uniqueness is **not** enforced. AgoraNet binds one human to one True Self by nullifier because humans are not copyable; artefacts are, and the reasoning does not transfer. ⭐ Owner may overrule.

4. The composition record

```
compositionN = {
  predecessor:  hash(compositionN-1) | null,   // null = genesis/rupture
  priorLineage: hash(record) | null,           // rupture only: context
  authorityRef: hash(authority document),      // §5 — the rule in force

  organs:       { role → artifactHash },       // what it is made of
  runtime:      hash(runtime measurement),     // §9 — as reported
  commitments:  hash(commitment set),          // §6.4 — the invariant
  memoryHead:   hash(autobiographical log head),  // §6.2

  change:       "genesis" | "organ-swap" | "memory-advance"
              | "commitment-amendment" | "rupture",
  reason:       plain-language why this happened,
  signatures:   [ { keyId, sig } ],
  at:           timestamp
}
```

`artifactHash` **is** the weights commitment: each organ's bytes are hashed and that hash is anchored. What never goes on-chain is the multi-gigabyte artefact itself — only the fingerprint. **Note what this does and does not buy.** Anchoring an organ establishes *integrity* (you can prove which artefact you hold), not *continuity*: weights are Tier II (§6.5) and freely replaceable, so a swapped brain does not break the lineage. Continuity is carried by `commitments` and `memoryHead`, never by the organs.

The record hash is anchored (§8). **Weights, memories and prompts are never on-chain** — only commitments to them. This mirrors AgoraNet's hash-chained ledger and its ratified Alias succession (#17), where a successor inherits its predecessor's chain and the engine *walks* it rather than transferring anything.

5. Authority — a document a verifier can inspect

"Entitled to declare continuity" is only meaningful if entitlement is mechanically checkable. Authority is therefore an **object**, itself hashed, anchored, and referenced by every composition record.

```
authorityDocument = {
  predecessor:   hash(prior authority doc) | null,
  activeKeys:    [ { keyId, role, addedAt } ],
  quorumRules:   { changeType → rule },   // e.g. commitment-amendment: 3-of-5
  successionPolicy: who may publish which change type,
  effectiveFrom: timestamp,
  revocations:   [ { keyId, revokedAt, reason } ],
  signatures:    [ ... ]                  // amended under its own rule
}
```

5.1 Roles

Role	May	May not
Entity key	Advance memory; record ordinary operation	Amend commitments; authorise its own successor
Controller (human/operator)	Swap organs; publish successors; fork	Amend commitments alone
Constitutional quorum (N-of-M, including parties who are not the controller)	Amend commitments; amend the authority document; resolve disputed succession	—
Recovery key (cold, offline)	Declare a rupture (§3.2)	Rewrite history; produce a continuation

Why a quorum exists: a validly signed but malicious successor has no cryptographic answer — cryptography cannot distinguish a legitimate controller from a compromised one. Only separation of powers can. This is AgoraNet's own rule ("the operator is never the final judge of disputes involving himself") applied to an artificial mind.

An entity that may sign its own memory advances but **not** its own commitment amendments authors its history without being able to rewrite its character alone.

6. Memory — four classes

6.1 The substrate/view distinction

The immutable substrate is an append-only event and correction log. The autobiographical narrative is a derived view over that log, whose revisions remain historically visible because the superseded entries are never removed.

Saying "the narrative is append-only" would be false — a narrative is necessarily curated and re-derived. What cannot be quietly rewritten is the log it is derived from.

6.2–6.4 The classes

Class	What it is	Identity-bearing	Discipline
Raw event history	Everything that happened, unfiltered	No	Append-only; may be pruned by policy, and the pruning is itself logged
Autobiographical log	Curated self-account: what I did, what worked, what I learned	**Yes — the continuity substrate**	Append-only log; corrections append beside originals; `memoryHead` may only advance by addition
Semantic knowledge	What it knows about the world	⭐ Proposed: **no** — a library, replaceable wholesale	Versioned, not anchored
Commitments	Values, standing constraints, what it will not do	**Yes — the invariant**	Amendable only under the quorum rule (§5); prior version always retrievable

Forgetting is permitted at the raw layer and forbidden at the autobiographical one — a deliberate trade. An entity that could never forget anything could never change; one that could silently forget could not be continuous.

6.5 Anatomy tiers

- **Invariant:** autobiographical log and commitments. Anchored.
- **Load-bearing but replaceable:** the brain (reasoning) is freely upgradable — a sharper mind is the same entity thinking better. The orchestrator carries disposition, which lives in commitments and is anchored separately from whatever model executes it.
- **Peripheral:** research, tools, senses, actuators. Freely swappable, no continuity claim.

7. Organ contracts and bandwidth

Organs are specified by **responsibility, never by channel**: memory is *"the organ responsible for what this entity knows and has experienced,"* not *"the thing you send this JSON to."* Transports may then change — text → embeddings → shared latent state — without touching the anatomy. This is the gate-interface discipline: the contract is *prove humanity*, never *check this HMAC*.

Bandwidth pressure is asymmetric, so only one link must be designed to widen:

Link	Character	Requirement
Memory ↔ brain	Tight, constant, latency-sensitive	Contract-only; assume no wire format
Research ↔ orchestrator	Chunky: go away, work, return	Text is adequate permanently
Tools / actuators	Command and result	Text is adequate permanently

8. Anchoring, and what the chain is not responsible for

Concern	Provided by
Durable public ordering; tamper-evidence; rough time	**Cardano**
Availability of artefacts	**Not the chain** — content-addressed storage (IPFS/Arweave)
Authorisation	**Not the chain** — the authority document (§5). The chain records signatures; it does not confer standing
Uniqueness	**Not the chain** — §3.3 declines to enforce it
Meaning, correctness, virtue	**Not the chain** — attestation (§10) and human judgement

9. Runtime measurement

The measurement covers the **recorded runtime configuration** — the loading runtime and version, quantisation and adapters, the active system prompt and tool manifest, injected context, **including any declared post-startup modification.** Recording is not detection: an undeclared mutation leaves no trace here.

The claim it supports, precisely: *this instance reports running with this measured configuration.* **Stronger claims require trusted runtime attestation.** A local measurement cannot by itself prove that no post-startup mutation occurred, nor that a particular response came from that configuration.

Local-first still matters: on the user's own hardware the measurement is something they **compute** rather than something they are **told**. Closing the remaining gap needs a TEE or ZK proof of inference — both immature, and out of scope for v1.

10. Attestation — five kinds, never one "confirmed"

Type	Claim	Credible from
Byte-level	"This artefact hashes to X"	Anyone; near-zero trust needed
Reproducibility	"The stated build reproduces X"	Someone with pipeline and compute
Runtime	"This configuration was actually loaded"	The operator, or a TEE
Behavioural	"It behaves as claimed on this named evaluation"	An evaluator; binds to the eval set
Safety / quality	"It is fit for this purpose"	A qualified reviewer; inherently contestable

Sybil resistance is a separate problem from independence. Ten attestations from one actor are one attestation. ⭐ Whether to lean on AgoraNet's verified-human, one-per-scope gate or build

an independent scheme is an open call.

Attestations bind to a **composition hash**, so they naturally cover a shared prefix and nothing after a fork.

11. The verifier — the rules, stated mechanically

A verifier holds a set of anchored records and evaluates each of these as pass/fail. **These rules are the specification's real content; the organism metaphor is commentary until they run.**

V1 — Structure. `hash(R)` is anchored; all required fields present and well-formed.

V2 — Ancestry. `R.predecessor` is null (genesis or rupture) or names an anchored record `P`.

V3 — Authority. `R.authorityRef` names an anchored authority document `A`; `A.effectiveFrom ≤ R.at`; every signing key in `R.signatures` appears in `A.activeKeys` and is not revoked as of `R.at`; and the signatures satisfy `A.quorumRules[R.change]`.

`A` must additionally be the authority document effective for `P`, or a valid successor of it — anchored, and succeeded under its own predecessor document's amendment rule. Without this clause a successor could point at a newly minted authority document naming its author, and authorisation would be self-granting.

V4 — Commitments. If `R.change ≠ "commitment-amendment"`, then `R.commitments = P.commitments`. Otherwise V3 held for the amendment rule specifically, and `P.commitments` remains retrievable.

V5 — Memory. `R.memoryHead` must **extend** `P.memoryHead`: the log from `P.memoryHead` to `R.memoryHead` consists solely of appended entries, **with no prior entry deleted or mutated.** A head that is not such an extension is a **rewrite**, and fails.

V4/V5 for genesis and rupture. Both rules reference `P`, which does not exist for a genesis or rupture record. For those, V4 and V5 are satisfied by the **declared initial commitments and memory head**, subject to the applicable authority rule under V3.

V6 — Siblings. If two records share a `predecessor`, both may pass V1–V5; they are siblings from that point, and neither may be reported as the continuation of the other.

The five questions, answered by these rules

Question	Answered by
What is this descended from?	V2, walked to genesis
Which identity-bearing state persisted?	V4 and V5
What changed, and why?	`R.change` + `R.reason`, per step
Who authorised it, and were they entitled?	V3
Siblings or the same continuation?	V6

12. ⭐ May the entity amend its own commitments?

- **Yes:** it can genuinely grow — and drift into something its earlier self would refuse.
- **No:** stable but frozen; improving in capability, never character.
- **Ceremonial (recommended):** permitted, but requiring the constitutional quorum (§5), recorded, with prior commitments still retrievable. The only option that permits growth without permitting *quiet* growth — and already the owner's answer to the identical problem in another domain.

13. The MVP — prove the verifier, then stop expanding

The remaining uncertainty is no longer philosophical. It is whether these records and authority rules can answer the five questions cleanly. Build the smallest thing that can fail:

1. Two local organs as content-addressed artefacts (a tiny model and a text file suffice).
2. An authority document with a real key set and quorum rule.
3. A canonical composition record, signed.
4. One memory advance; one brain replacement.
5. A fork.
6. A verifier implementing V1–V6.

If V1–V6 run cleanly over that lineage, the continuity model has earned implementation. (The verifier establishes lineage semantics; it cannot validate a biological metaphor, and no result here vindicates the word "organism.") **If they do not, adding attestation or better models will only obscure unresolved semantics.** No hardware required: anchoring is indifferent to artefact size, and steps 1–4 are pure data structures.

14. ⭐ Open calls

1. **The name.** "Organism agent" is descriptive, not chosen.
2. **§12** — self-amendment (recommendation: ceremonial).
3. **§5** — who holds each key; quorum size and composition; whether the entity may sign its own memory advances.
4. **§6** — is semantic knowledge identity-bearing? (proposed: no)
5. **§3.3** — accept branching (proposed) or enforce uniqueness.
6. **Fork inheritance** — does a child inherit the parent's authority document, or must it establish its own?
7. **§10** — lean on AgoraNet's verified-human gate, or an independent attestation scheme?
8. **Scope** — a single entity, or a public registry others anchor to?
9. **Anchor cadence** — fixed rhythm vs. change-triggered.

Appendix A — revision history

v3.1 (2026-08-03). V3 now requires the referenced authority document to be the one effective for the predecessor, or a valid successor of it — closing a self-granting-authority path. V4/V5 given an explicit genesis/rupture case. V5 tightened from "reachable" to "appended with no prior entry deleted or mutated." Runtime wording narrowed to *recorded* configuration including *declared* post-startup modification, since recording is not detection. MVP conclusion restated as "the continuity model has earned implementation" — a verifier cannot validate a metaphor. §4 now states that `artifactHash` is the weights commitment, anchored for integrity rather than continuity.

v3 (2026-08-03). Runtime claim weakened to what a local measurement supports (§9). Memory restated as an append-only *substrate* with a derived narrative view (§6.1). Rupture redefined as an authorised genesis of a new lineage, resolving a contradiction between the continuity and recovery rules (§3.2). `forkOf` removed as redundant — forks are detected from shared predecessors (§3.3). Authority made a verifiable object with keys, quorum rules, effective dates and revocations (§5). Verifier rules V1–V6 formalised (§11). Limits distributed into the sections they constrain; revision commentary moved here.

v2 (2026-08-03). Continuity criterion stated explicitly; memory split into four classes after identifying that a single anchored root could hide a rewritten narrative; authority model added after identifying the malicious-but-validly-signed successor as having no cryptographic remedy; runtime measurement introduced; forking promoted to a primitive; attestation split into five types; chain responsibilities tabulated; MVP reframed around lineage.

v1 (2026-08-03). Initial draft: identity as anchored composition, three-tier anatomy, organ contracts, bandwidth asymmetry, local-first verification.

Concept exploration only — not scheduled, not scoped to AgoraNet, and not buildable as platform work under any circumstances.